9 Recurrent Neural Networks

Chapter Abstract. A review model can look faster because it truncated the sentence that changed the verdict, or look slower because half the batch is padded token slots. This chapter introduces recurrent neural networks as models for ordered data through binary sentiment classification on IMDB movie reviews: each review is a sequence of token IDs, and the model predicts whether the review is positive or negative. The chapter defines the simple Elman RNN, unrolling through time, dynamic sequence handling, padding, masks, collation, bucketing, long-dependency failures, LSTM cells, peephole LSTM variants, GRUs, gating as a broader design pattern, recurrent dropout, normalization choices for recurrent models, and fast.ai's AWD-style LSTM recipe. The practical emphasis is on a reproducible sequence-classification experiment that measures length policy, padding waste, validation behavior, runtime, and final-test discipline together.

9.1 A Running Sequence Example

The running example is sentiment analysis on the IMDB movie-review dataset introduced by Maas and colleagues [175]. A review is text, and text has order. The phrase "not good" is different from "good, not great" even though the two examples share words. The chapter begins here because this ordered evidence is exactly what a feedforward bag-of-words classifier tends to hide. Using the token-ID and embedding-matrix vocabulary from Chapter 8, a review becomes a finite ordered sequence

$$(x_1, x_2, \ldots, x_T),$$

where T is the length of this particular review. Each position keeps its place: after embedding, token position t contributes one vector in $\mathbb{R}^D$ rather than being immediately pooled away.

The label for an IMDB review is binary: positive or negative. The model therefore reads the input sequence and eventually produces one binary sentiment logit, interpreted with the logits-and-predictions convention from earlier classifier chapters. This is a many-to-one sequence task because many input positions produce one output label. Other sequence tasks have different shapes. Speech recognition and translation are many-to-many tasks; a weather model that predicts tomorrow from the last thirty days is also many-to-one; a music generator that starts from one prompt and emits many notes is one-to-many. The common feature is that an example is not a fixed unordered vector but an ordered record.

The practical goal is not merely to name an architecture. A useful sequence experiment must decide how long the examples are allowed to be, how short examples are padded, how padding is masked, how batches are formed, which recurrent cell is used, how gradients are stabilized, and

when the official test set is evaluated. The chapter therefore treats RNNs as both mathematical functions and training systems. The mathematics explains what the model computes; the data pipeline determines whether that computation is actually applied to the intended tokens.

A recurrent neural network is one way to make the order in that running example part of the computation. It is often drawn as a neural network with a loop. That drawing is a useful first memory aid, but it is not yet a mathematical definition. In this chapter an RNN is a neural network that applies the same state-update function repeatedly along an ordered sequence. At step t, the model receives the current input vector x_t and the previous hidden state h_{t-1}, then produces a new hidden state h_t. The loop in the compact drawing means that the output state from one step becomes an input to the next step. When the model is trained, the loop is unfolded into an ordinary feedforward computation graph with one copy of the cell for each sequence position.

Figure 9.1 puts those two views side by side. The compact loop is useful notation; the unrolled graph is the object that backpropagation traverses for a concrete sequence length.

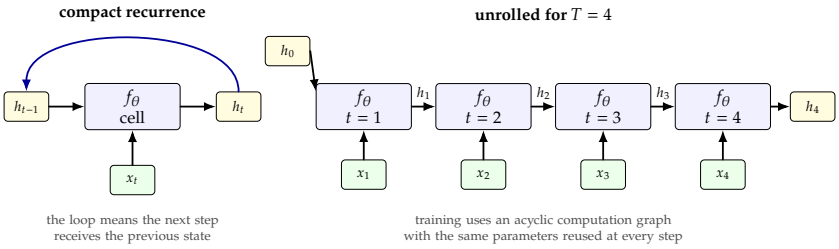

Figure 9.1: Compact and unrolled views of the same recurrent computation. The compact loop is notation for repeatedly applying the same state-update function; for a fixed sequence length, the training graph is unrolled through time.

Pause and predict

Suppose an IMDB mini-batch has shape $(32, 200, 64)$ after embedding. Which axis should an RNN scan as the ordered sequence, and which axis contains the vector features at one position?

9.2 Sequence Data And Sequence Tasks

A sequence is a finite ordered tuple $(x_1, \ldots, x_T)$. The index t is called a time step even when the sequence is not literally time. In text, t is usually a token position. In a sound recording, t may index short audio frames. In a medical record, t may index visits. The important point is that exchanging two positions can change the example. A feedforward network can still receive a flattened sequence, but flattening hides the regular structure that one position follows another.

Figure 9.2 shows several examples before the notation becomes specific to movie reviews. The common feature is not the domain itself, but the fact that each example is an ordered record.

Figure 9.2: AI-generated conceptual illustration of sequence data in several domains. Text tokens, audio frames, weather observations, and medical visits can all be represented as ordered records before a model chooses an appropriate sequence task.

For the IMDB example, each raw review is a string. As in Chapter 8, preprocessing maps words or subword pieces to token IDs and an embedding table maps each ID to a vector. The new rule is that the sequence is not pooled immediately. If $B = 32$ reviews are padded to a common length $T_{\max} = 200$, the embedded batch has shape $\mathbb{R}^{32\times200\times D}$: batch item, token position, and embedding feature.

Sequence tasks are classified by the relationship between input and output positions. In many-to-one classification, the model predicts one label after reading the sequence. IMDB sentiment classification is of this kind. In many-to-many labeling, the model predicts one label for each input token; part-of-speech tagging is an example. In sequence-to-sequence prediction, the output is also a sequence, but its length may differ from the input length, as in translation. Figure 9.3 summarizes these common shapes. This chapter uses many-to-one classification because it keeps the code and evaluation simple while still exposing the central sequence issues.

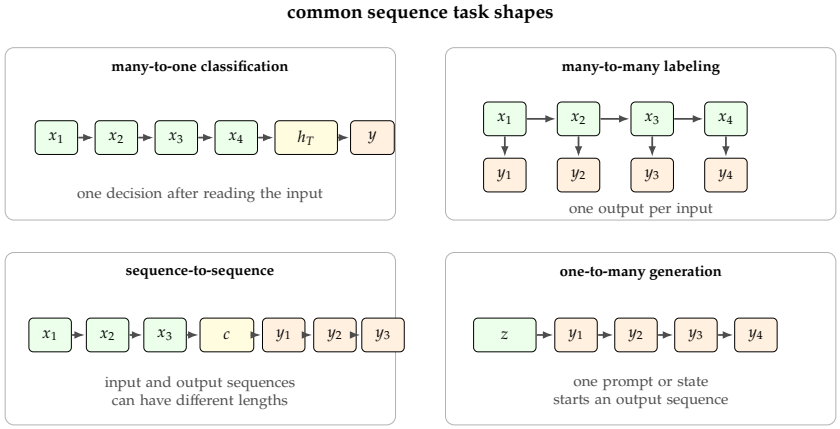

Figure 9.3: Common sequence task shapes. IMDB sentiment classification is many-to-one, but recurrent and attention-based sequence models also appear in aligned labeling, sequence-to-sequence prediction, and generation settings.

The first applied habit is to compare against a simple non-recurrent baseline. For sentiment analysis, a baseline might average token embeddings and classify the average. Such a model ignores word order, so it cannot

represent every distinction that an RNN can represent. Still, it may train quickly and perform surprisingly well on short reviews where individual sentiment words are strong. If a recurrent model is slower and not more accurate on validation data, the name of the architecture is not a sufficient reason to prefer it.

The second applied habit is to inspect lengths before choosing a maximum length. IMDB reviews vary widely. If a model keeps only the first 80 tokens, it may train quickly but discard useful evidence. If it keeps 1000 tokens for every review, many batches may spend most of their computation on padding. The right choice is an experimental design decision: inspect the length distribution, choose a policy such as 200 or 400 tokens for the first run, and record that policy with the result.

9.3 The Simple Elman RNN Cell

The basic recurrent cell in this chapter is the Elman, or simple, RNN cell [50]. Let $x_t \in \mathbb{R}^D$ be the input vector at position t, and let $h_t \in \mathbb{R}^H$ be the hidden state after the model has read positions 1 through t. The hidden width H is a modeling choice. With initial state $h_0 = 0$, a simple RNN computes

State size. H controls how much information the recurrent layer can carry forward and how many recurrent parameters it uses.

$$a_t = W_{xh}x_t + W_{hh}h_{t-1} + b_h, \qquad h_t = \phi(a_t),$$

where $W_{xh} \in \mathbb{R}^{H\times D}$, $W_{hh} \in \mathbb{R}^{H\times H}$, $b_h \in \mathbb{R}^H$, and ϕ is usually tanh or another nonlinear activation. A many-to-one sentiment classifier can then use the final valid hidden state:

$$z = w_y^\top h_T + b_y,$$

where $z \in \mathbb{R}$ is the sentiment logit.

Figure 9.4 shows the same update as a cell diagram. The important point is that the input path and recurrent path meet before the nonlinearity, and the resulting hidden state is the state carried to the next position.

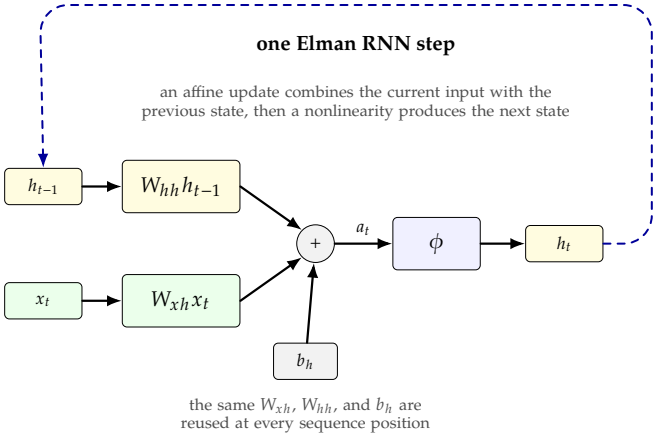

Figure 9.4: Simple Elman RNN cell. The current input and previous hidden state are mapped into one preactivation, a nonlinearity produces h_t, and that hidden state is passed to the next time step.

Pause and predict

Let $D = 64$ and $H = 128$. Before reading on, predict the shapes of W_{xh}, W_{hh}, b_h, h_t, and the sentiment logit z.

The answers are $W_{xh} \in \mathbb{R}^{128\times 64}$, $W_{hh} \in \mathbb{R}^{128\times 128}$, $b_h \in \mathbb{R}^{128}$, $h_t \in \mathbb{R}^{128}$, and $z \in \mathbb{R}$.

Memory is learned. The state is not a tape; it is a compressed representation trained for the task.

The hidden state is sometimes described as memory, but the precise statement is that h_t is a learned vector-valued function of the prefix $(x_1, \ldots, x_t)$. The recurrence makes this possible because h_t depends on x_t and h_{t-1}, while h_{t-1} already depends on earlier inputs. In the phrase "not a good movie," the state after reading "good" can still be influenced by the earlier token "not." Whether training actually learns such a useful representation is an empirical question.

A scalar example shows the arithmetic. Suppose $D = H = 1$, $h_0 = 0$, $\phi = \tanh$, $w_x = 1.2$, $w_h = 0.5$, and $b = 0$. Encode three token features as $x_1 = 0.6$, $x_2 = -0.4$, and $x_3 = 0.8$. Then

$$h_1 = \tanh(1.2 \cdot 0.6 + 0.5 \cdot 0) = \tanh(0.72) \approx 0.617.$$

The second state uses the previous state:

$$h_2 = \tanh(1.2 \cdot (-0.4) + 0.5 \cdot 0.617) = \tanh(-0.1715) \approx -0.170.$$

The third state is

$$h_3 = \tanh(1.2 \cdot 0.8 + 0.5 \cdot (-0.170)) = \tanh(0.875) \approx 0.704.$$

Even in this one-dimensional toy model, the third output is not a function of x_3 alone. It is affected by the earlier tokens through h_2.

Shared transition rule. The recurrent weights are reused at every position, so parameter count does not grow with sequence length.

RNN weight sharing is the sequence analogue of the CNN weight sharing introduced in Chapter 4. A simple RNN applies the same matrices W_{xh} and W_{hh} at many sequence positions. The reason for sharing is not identical: CNNs assume that a local visual pattern can be useful at different image locations, while RNNs assume that the same transition rule can be reused at different time or token positions. In both cases, sharing keeps the parameter count independent of the number of positions.

For a tensor-shape check, let a padded mini-batch contain $B = 32$ reviews, each represented with maximum length $T_{\max} = 200$ and embedding dimension $D = 64$. The input to the recurrent layer has shape $(32, 200, 64)$ in the common batch-first convention. If the hidden width is $H = 128$ and the layer returns every hidden state, the output sequence has shape $(32, 200, 128)$. If the layer returns only the last valid state for each review, the classifier receives a tensor with shape $(32, 128)$.

An RNN model can also have multiple recurrent layers stacked on top of one another. In a stacked RNN, the first recurrent layer reads the input sequence and produces hidden states across time. The next recurrent layer reads that hidden-state sequence as its own input sequence:

$$h_t^{(1)} = f_{\theta_1}(x_t, h_{t-1}^{(1)}), \qquad h_t^{(\ell)} = f_{\theta_\ell}(h_t^{(\ell-1)}, h_{t-1}^{(\ell)}) \quad \text{for } \ell = 2, \dots, L.$$

This creates two different dimensions of depth, shown in Figure 9.5: sequence time runs horizontally through each recurrent layer, while ordinary neural-network depth runs vertically from lower recurrent layers to higher recurrent layers. In tensor terms, a lower recurrent layer must return the whole sequence, such as (B, T, H_1), so that the next recurrent layer has one input vector at every time step. The top recurrent layer may return the whole sequence for a many-to-many task or only the final valid state for a many-to-one classifier.

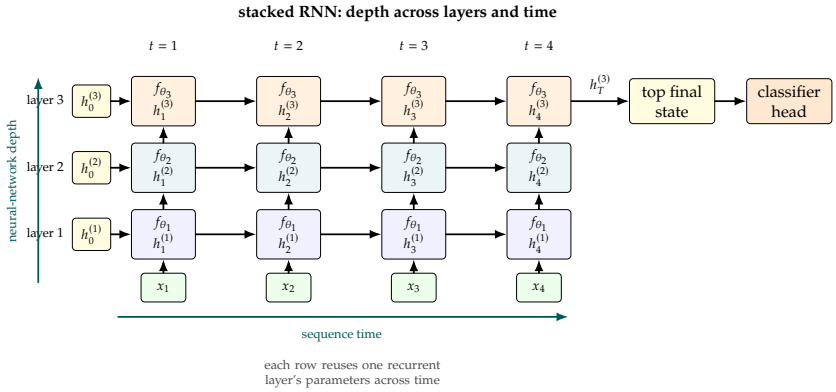

Figure 9.5: A stacked recurrent network unrolled through time. Horizontal connections carry each layer's hidden state forward through the sequence, while vertical connections feed the hidden state at each time step into the next recurrent layer. The model therefore has depth across neural-network layers and length across sequence time.

The simple RNN is a valuable teaching baseline, but it is rarely the first practical choice for text classification. Library implementations of LSTM and GRU layers usually provide better behavior on longer dependencies with little extra code. A disciplined first experiment can still include a simple RNN, because it gives a controlled comparison: the data pipeline, embeddings, optimizer, and evaluation protocol remain the same while only the recurrent cell changes. Stacking recurrent layers is a later architecture experiment, not a substitute for first choosing a cell and validating a single-layer baseline.

9.4 Unrolling Through Time And Dynamic RNNs

The loop in an RNN diagram is compact notation. For a concrete sequence of length $T = 4$, the computation is

$$h_1 = f_\theta(x_1, h_0), \quad h_2 = f_\theta(x_2, h_1), \quad h_3 = f_\theta(x_3, h_2), \quad h_4 = f_\theta(x_4, h_3),$$

where the same parameter set θ is used in every copy of the cell. This expanded view is called unrolling through time. After unrolling, the graph has no directed cycle for this fixed input length, so the computation-graph and backpropagation machinery from Chapter 5 can be applied to it.

The right side of Figure 9.1 is the graph behind this equation: four applications of the same cell, connected by the hidden states.

Backpropagation through time, or BPTT, names that earlier backpropagation procedure when it is run on the unrolled sequence graph [299, 72]. If a loss $\mathscr{L}$ is computed from the final state h_T, the gradient of $\mathscr{L}$ with respect to W_{hh} receives contributions from every time step where W_{hh} was used. This is different from having separate weights at each position. The parameter is shared, so its update combines evidence from position 1, position 2, and so on. The price is memory: during training, intermediate states are usually needed for the backward pass. Longer sequences therefore cost more memory and more time.

BPTT cost. Training usually stores the unrolled states needed for the backward pass, so long sequences are expensive.

The shared-parameter gradient can be written as a sum over time:

$$\frac{\partial \mathscr{L}}{\partial W_{hh}} = \sum_{t=1}^{T} \frac{\partial \mathscr{L}}{\partial h_t} \frac{\partial h_t}{\partial W_{hh}}.$$

The sensitivity $\partial \mathscr{L} / \partial h_t$ itself carries chain-rule factors through later states $h_{t+1}, \ldots, h_T$. This is the mathematical reason long sequences can create vanishing or exploding gradients: many Jacobians are multiplied along the temporal path before their contributions reach early time steps.

In very long sequences, one may use truncated BPTT. Instead of backpropagating through the entire history, training is broken into shorter windows. This reduces memory and can make training feasible, but it also limits the gradient path used to assign credit across distant positions. Figure 9.6 contrasts the full and truncated cases. In the IMDB homework, truncation is usually less important than choosing a reasonable maximum review length, because the examples are finite reviews rather than continuous streams.

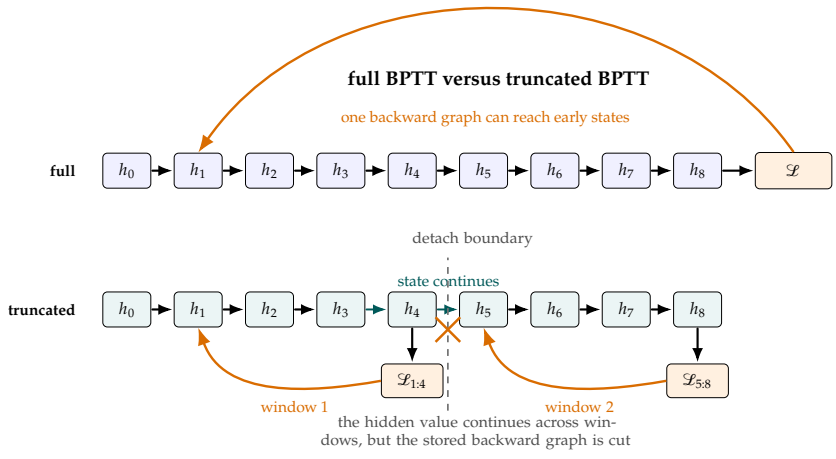

Figure 9.6: Backpropagation through time on an unrolled recurrent graph. Full BPTT lets a final loss send gradients through the entire stored sequence. Truncated BPTT processes shorter windows, carrying the hidden state value forward while stopping gradients at window boundaries.

The term dynamic RNN is often used in libraries and discussions, but it should be interpreted carefully. It does not mean that the model magically ignores all data-pipeline decisions. It usually means that the recurrent computation can be run for the actual sequence length at runtime, or that the framework can use masks or packed sequences to avoid treating padded positions as real observations. A dynamic implementation can handle

Padding is not data. A classifier should use the final valid token state, not a state created only from padding.

different review lengths, but the mini-batch still needs a representation: padded tensors with masks, ragged tensors, or packed sequences.

PyTorch's packed-sequence utilities are one explicit approach: sequences are sorted or length-described, packed, processed by recurrent modules, and then unpacked when padded tensors are needed again [216]. Keras commonly handles beginner sequence models by padding integer sequences and using masking, for example through an embedding layer with `mask_zero=True` when the downstream recurrent layer supports masks [133, 141]. These are implementation details, but they reflect the same mathematical requirement: the model must know which positions are real tokens.

Pause and predict

If a batch is padded to length 200 but one review has true length 37, should the final-state classifier use the state at position 37 or the state at position 200?

It should use the final valid state, position 37, or an equivalent masked computation. Position 200 is a padding position for that review.

9.5 Padding, Collation, And Bucketing

Real sequence datasets do not arrive as rectangular arrays. One IMDB review may contain 35 tokens and another may contain 600. Hardware kernels, however, are most efficient on regular tensors. The text padding used in Chapter 8 now becomes part of the recurrent pipeline: within a mini-batch, shorter sequences receive a special pad token until every example has the same length. A mask or a length vector records which positions are real. Unlike convolutional boundary padding from Chapter 4, these pad tokens are artificial sequence positions and must not be treated as normal words.

Consider three short tokenized reviews:

"this film was good"	$(12, 48, 23, 77)$,
"not good"	$(95, 77)$,
"slow but moving"	$(310, 44, 822)$.

Pause and predict

Before reading the table, pad these three reviews to the longest length and write the matching mask. Which positions should contain the pad token, and which mask entries should be 0?

The longest length is 4, so padding with token ID 0 gives

review	padded token IDs				mask			
1	12	48	23	77	1	1	1	1
2	95	77	0	0	1	1	0	0
3	310	44	822	0	1	1	1	0

The padded token array has shape $(3, 4)$ before embedding. The mask has the same first two dimensions. It says that the last two positions of review 2 and the last position of review 3 are not review content.

Figure 9.7 redraws the same idea as tensor slots. The shaded cells are useful because they make the waste calculation visible before it becomes a formula.

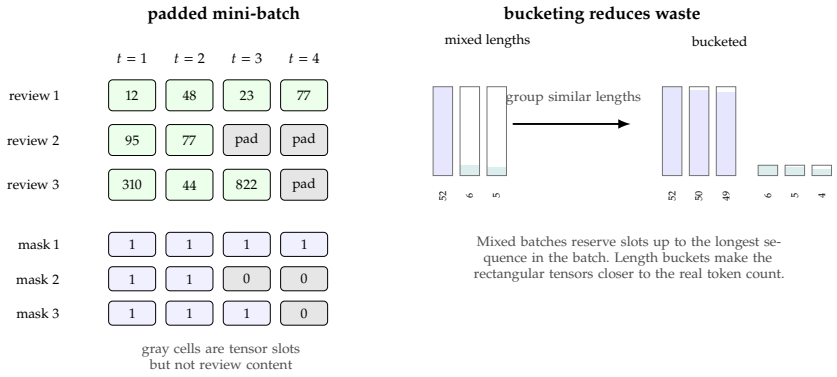

Figure 9.7: Padding turns variable-length reviews into rectangular tensors, and the mask identifies real token positions. Bucketing is a batching strategy: reviews of similar length are grouped so fewer tensor slots are wasted on padding.

Collation is the step that builds this mini-batch from individual examples. A dataset may store each review as a separate list of token IDs. A collate function receives a list of examples, finds their lengths, pads them, builds labels, and often returns a length vector or mask. In Keras, a preprocessing call such as `pad_sequences` performs the padding for a whole array of examples. In direct PyTorch code, collation is often attached to a `DataLoader` [217]. The principle is the same: the model should receive both the rectangular tensor and enough information to distinguish tokens from padding.

Padding also affects speed. Sequence models pay for time steps, including empty ones, so padding waste is not a cosmetic detail in the batch printout. In the three-review batch above, the rectangular tensor has $3 \cdot 4 = 12$ token slots, but only $4 + 2 + 3 = 9$ are real. The padding waste is therefore $(12 - 9)/12 = 0.25$, or 25%. That small example is harmless. In a larger batch with lengths $(600, 40, 38, 37)$ padded to 600, the tensor has 2400 slots and only 715 real tokens, so about 70.2% of the positions are padding. Training will be slower, and an incorrect mask can also make the model learn from artificial zeros.

Padding waste. Long outliers can make a batch mostly padding; masks protect correctness, while bucketing protects throughput.

Bucketing reduces padding by grouping sequences of similar length before batching. Suppose six reviews have lengths $(52, 50, 49, 6, 5, 4)$. If one batch contains $(52, 6, 5)$, it has $3 \cdot 52 = 156$ slots but only 63 real tokens. If another batch contains $(50, 49, 4)$, it has 150 slots but only 103 real tokens. Together the two batches waste $(306 - 166)/306 \approx 45.8\%$ of the slots. If the batches are length-bucketed as $(52, 50, 49)$ and $(6, 5, 4)$, they use $156 + 18 = 174$ slots for the same 166 real tokens, wasting only about 4.6%.

Bucketing is not a change to the RNN cell. It is a batching strategy that can improve throughput. It can also make training curves less noisy when batches have more consistent shapes. The tradeoff is that completely sorting the dataset by length can reduce randomness, so practical input pipelines usually shuffle within buckets or create buckets stochastically.

For a beginner IMDB experiment, padding to a fixed maximum length is acceptable. For larger sequence projects, the padding ratio should be recorded and bucketing should be considered when the ratio is high.

9.6 Why Simple RNNs Struggle

The simple RNN cell can represent dependencies across positions in principle, but it is difficult to train on long dependencies in practice. The backward signal from a late loss must pass through many repeated transformations before it reaches early positions. Repeated multiplication can make gradients shrink toward zero or grow to unstable magnitudes. This difficulty was identified early in the study of recurrent networks and remains a central reason to use gated cells such as LSTMs and GRUs [12, 209].

Pause and predict

Suppose a backward signal is multiplied by the same scalar at each earlier time step. Before reading the calculation, predict what happens after 10 steps with multiplier 0.5, and what happens with multiplier 1.5. Which case suggests vanishing gradients, and which suggests exploding gradients?

A scalar calculation captures the intuition. If a gradient is multiplied by 0.5 at each step, then after 10 steps its multiplier is

$$0.5^{10} = 0.0009765625.$$

An update signal of size 1 at the end of the sequence has become less than 0.001 by the time it reaches a token ten positions earlier. If the multiplier is 1.5 instead, then

$$1.5^{10} \approx 57.7.$$

Now a modest signal can become very large. Real RNNs use matrices and nonlinearities, not one scalar multiplier, but the same repeated-product problem appears through the recurrent Jacobians.

Figure 9.8 visualizes the two scalar cases. The point is not the exact numbers themselves, but the speed with which repeated multiplication can erase or amplify a signal.

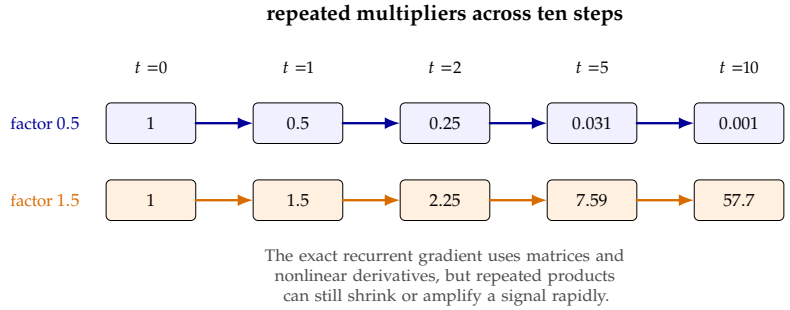

Figure 9.8: Toy repeated-gradient multipliers. A factor below one quickly erases a backward signal, while a factor above one can make it unstable. This figure illustrates the arithmetic behind vanishing and exploding gradients.

The tanh activation can make vanishing gradients worse when its input is large in magnitude. Near zero, tanh changes quickly. Far from zero, it saturates near -1 or 1, and its derivative becomes small. A simple RNN that repeatedly pushes hidden units into saturated regions may preserve a forward state value but still provide weak gradients for learning how earlier tokens should affect later predictions.

In IMDB sentiment analysis, long-distance dependencies appear in ordinary language. The sentence "I expected to hate this movie, but after the first hour I did not want it to end" cannot be classified by the word "hate" alone. A model must learn how later words revise earlier sentiment. Conversely, "not good" requires an early negation to influence the interpretation of a later positive word. Simple RNNs may learn some of these patterns for short contexts, but validation curves often reveal that they are weaker than gated alternatives under the same training budget.

Exploding gradients have a direct engineering response: gradient clipping. Clipping limits the norm or value of gradients before the optimizer step [209]. It does not solve vanishing gradients, but it can prevent a training run from diverging when occasional long sequences create very large updates. For recurrent models, a sensible first recipe includes gradient clipping, a moderate learning rate, early stopping, and a comparison against GRU or LSTM cells.

The applied lesson is to avoid heroic tuning of a weak baseline. A simple RNN is useful because it makes recurrence transparent. If it learns slowly, overfits, or fails to improve over an average-embedding baseline, the next controlled change should usually be a GRU or LSTM, not an arbitrary stack of extra layers and dropout rates.

9.7 LSTM Cells: Gates And An Additive Path

Long short-term memory networks, or LSTMs, were introduced to make recurrent learning more effective over longer intervals [101]. The forget gate used in the modern form was introduced to let the cell reset or retain memory as needed [66]. A modern LSTM cell maintains two vectors. The hidden state $h_t \in \mathbb{R}^H$ is the exposed recurrent output. The cell state $c_t \in \mathbb{R}^H$ is an internal memory path. The crucial design is additive.

Additive path. The LSTM cell state can retain, erase, and add information with elementwise gates instead of fully rewriting memory at every step.

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t,$$

where $\odot$ denotes elementwise multiplication. The previous cell state is not simply passed through a nonlinear matrix multiplication. It is scaled by a learned forget gate f_t and then receives a learned write update $i_t \odot g_t$.

For input $x_t \in \mathbb{R}^D$, one common LSTM formulation is

$$
\begin{aligned}
i_t &= \text{sigmoid}(W_{xi}x_t + W_{hi}h_{t-1} + b_i),\\
f_t &= \text{sigmoid}(W_{xf}x_t + W_{hf}h_{t-1} + b_f),\\
g_t &= \tanh(W_{xg}x_t + W_{hg}h_{t-1} + b_g),\\
o_t &= \text{sigmoid}(W_{xo}x_t + W_{ho}h_{t-1} + b_o),\\
c_t &= f_t \odot c_{t-1} + i_t \odot g_t,\\
h_t &= o_t \odot \tanh(c_t).
\end{aligned}
$$

Equivalently, many implementations concatenate $[x_t, h_{t-1}]$, apply one learned affine projection to $4H$ numbers, and split the result into input-gate, forget-gate, candidate, and output-gate preactivations. In this notation g_t is candidate content, not a sigmoid gate. The input gate i_t controls how much new candidate content is written. The forget gate f_t controls how much previous cell content is retained. The output gate o_t controls how much of the cell state is exposed through the hidden state. Because sigmoid gate values lie between 0 and 1, they can be read as learned soft controls, not as hard if-statements.

Pause and predict

Before the numerical example, predict which gate should be near 1 to preserve old memory, which gate should be small to block a new write, and which gate controls how much of the cell state is exposed through h_t.

The answers are the forget gate f_t, the input gate i_t, and the output gate o_t.

Figure 9.9 emphasizes the additive cell-state path in the middle of these equations. That path is the main structural difference from the simple RNN update.

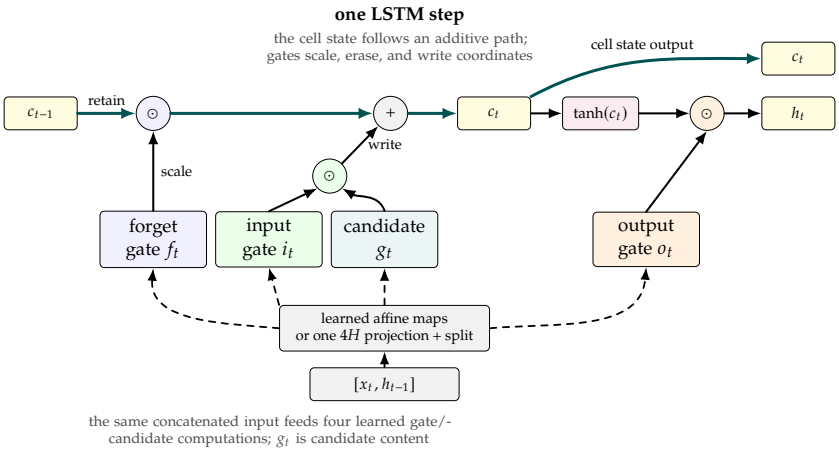

Figure 9.9: Simplified LSTM cell-state path. The concatenated vector $[x_t, h_{t-1}]$ feeds four learned gate/candidate computations, often implemented as one $4H$ projection that is split. The forget gate scales the old cell state, the input gate controls a new candidate write, and the cell returns both the updated cell state c_t and exposed hidden state h_t. The output gate controls how much of the cell state becomes h_t.

A one-dimensional example makes the additive path concrete. Suppose the previous cell state is $c_{t-1} = 1.0$. At the next token, let the forget gate be $f_t = 0.9$, the input gate $i_t = 0.2$, the candidate value $g_t = 0.5$, and

the output gate $o_t = 0.8$. Then

$$c_t = 0.9 \cdot 1.0 + 0.2 \cdot 0.5 = 1.0,$$

and

$$h_t = 0.8 \cdot \tanh(1.0) \approx 0.8 \cdot 0.762 = 0.610.$$

The cell retained most of the previous memory and added a small new contribution. If a later token should erase this memory, training can learn a smaller forget gate. If the memory should be preserved across many neutral tokens, training can learn forget gates close to 1.

The phrase "highway link" is an informal way to describe the additive cell-state path. It is not a separate layer in the sense of a highway network, but it plays a similar conceptual role: information can move through time by repeated additions and elementwise scaling rather than by being fully transformed at every step. This does not make LSTMs immune to all optimization problems, but it gives them a more direct route for preserving useful state.

In an IMDB classifier, an LSTM layer reads the embedded token sequence and produces either all hidden states or the final valid hidden state. The classifier head then maps the selected state to a sentiment logit. For a first serious recurrent baseline, a single LSTM layer with moderate hidden width, gradient clipping, and early stopping is often a better use of time than a deep stack of simple RNN layers.

9.8 Peephole LSTMs And GRUs

The word "peephole RNN" is usually better stated as peephole LSTM. In a peephole LSTM, gates can directly inspect the cell state in addition to the input and previous hidden state [67]. For example, a forget gate may include a learned peephole term such as $p_f \odot c_{t-1}$. The motivation is timing: if the cell state stores how much of some event has accumulated, a gate may benefit from seeing that internal value directly. Peephole connections are a variant of the LSTM idea, not a replacement for understanding the ordinary gated cell.

Figure 9.10 marks the extra connections with dashed arrows. The base $[x_t, h_{t-1}]$ gate and candidate maps are still present; peepholes add cell-state terms to selected gate computations rather than defining a new state update.

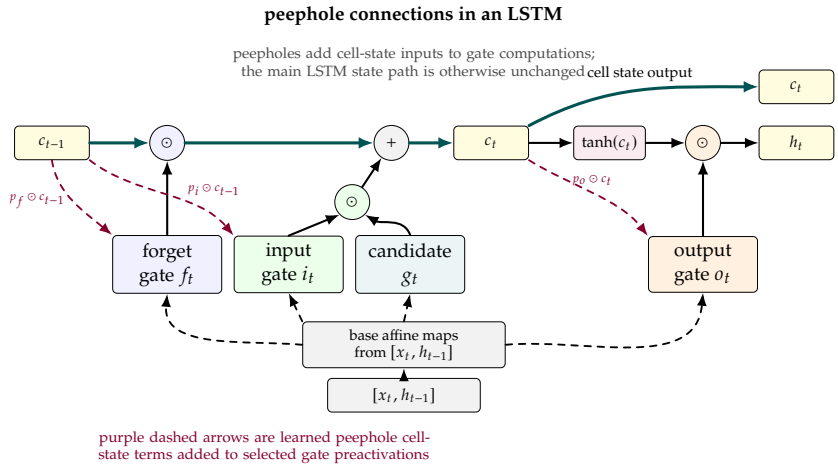

Figure 9.10: Peephole LSTM variant. The base $[x_t, h_{t-1}]$ maps still feed the gate and candidate computations. The extra connections let gates inspect the cell state directly by adding learned cell-state terms to forget, input, or output gate preactivations. The cell still returns both the updated cell state c_t and exposed hidden state h_t.

The gated recurrent unit, or GRU, is another widely used gated recurrent cell [28, 29]. A common GRU formulation uses an update gate z_t, a reset gate r_t, and a candidate hidden state n_t:

$$
\begin{aligned}
z_t &= \mathrm{sigmoid}(W_{xz} x_t + W_{hz} h_{t-1} + b_z), \\
r_t &= \mathrm{sigmoid}(W_{xr} x_t + W_{hr} h_{t-1} + b_r), \\
n_t &= \tanh(W_{xn} x_t + W_{hn}(r_t \odot h_{t-1}) + b_n), \\
h_t &= (1 - z_t) \odot n_t + z_t \odot h_{t-1}.
\end{aligned}
$$

The GRU has one state vector rather than separate hidden and cell states. Its update equation is also additive: it interpolates between a new candidate n_t and the previous state h_{t-1}.

Figure 9.11 makes that interpolation visible. The update gate controls the tradeoff between keeping the old state and writing the candidate, while the reset gate affects how the candidate is formed.

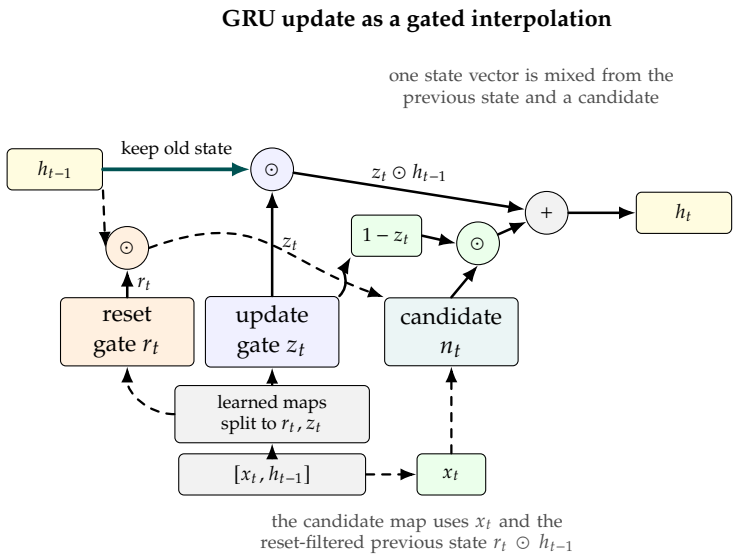

Figure 9.11: GRU state update schematic. The update gate controls how much previous state is retained versus replaced by a candidate, while the reset gate controls how strongly the previous state participates in forming that candidate. The candidate map uses x_t together with the reset-filtered previous state $r_t \odot h_{t-1}$.

Parameter counts help explain why GRUs can be attractive in applied work. Ignoring framework-specific choices about separate input and recurrent biases, a simple RNN with input dimension D and hidden width H has

$$H(D + H) + H$$

parameters. A GRU has three such groups, and an LSTM has four. With $D = 32$ and $H = 64$, the simple RNN count is

$$64(32 + 64) + 64 = 6208.$$

The GRU count is approximately $3 \cdot 6208 = 18624$, and the LSTM count is approximately $4 \cdot 6208 = 24832$. This is not a large number by modern standards, but it affects speed and memory when widths, layers, or vocabulary embeddings grow.

There is no universal rule that LSTM is always better than GRU or that GRU is always faster enough to prefer. The correct applied comparison fixes the data split, vocabulary, maximum length, embedding dimension, hidden width, optimizer, learning-rate policy, batch size, and epoch budget, then changes the recurrent cell. The result table should report validation accuracy, final test accuracy for the selected model, runtime, and parameter count. Architecture names alone are not evidence.

9.9 Gating Beyond Recurrent Cells

The LSTM and GRU sections introduced gates as learned soft controls inside a recurrent cell. The broader pattern is simpler than the recurrent setting:

$$\text{modulated output} = \text{signal} \odot \text{gate}.$$

The gate is usually a number or vector whose coordinates lie between 0 and 1, often produced by a sigmoid. What changes across architectures is where the gate comes from and what kind of information flow it controls.

For an LSTM or GRU, a typical gate is a learned function of the current input and the previous hidden state:

$$\text{recurrent gate}_t = \text{sigmoid}(W_x x_t + W_h h_{t-1} + b).$$

That gate is context-dependent and temporal. It can decide, coordinate by coordinate, whether the model should retain old state, write a candidate update, reset part of the previous state, or expose part of the state to the next layer.

Gating is not exclusive to recurrent networks. A squeeze-and-excitation block takes a convolutional feature tensor $X \in \mathbb{R}^{H \times W \times C}$, summarizes each channel with global average pooling,

$$z_c = \frac{1}{HW} \sum_{i,j} X_{i,j,c},$$

passes the channel summary through a small MLP and sigmoid,

$$s = \text{sigmoid}(\text{MLP}(z)),$$

and multiplies each original channel by its learned weight [109]:

$$\widetilde{X}_{i,j,c} = s_c X_{i,j,c}.$$

This is an input-dependent soft gate over channels. It is not an LSTM-style memory mechanism: there is no recurrent cell state and no temporal path to preserve. The same word *gate* is useful because the operation is still multiplicative control. Figure 9.12 summarizes the feedforward data path.

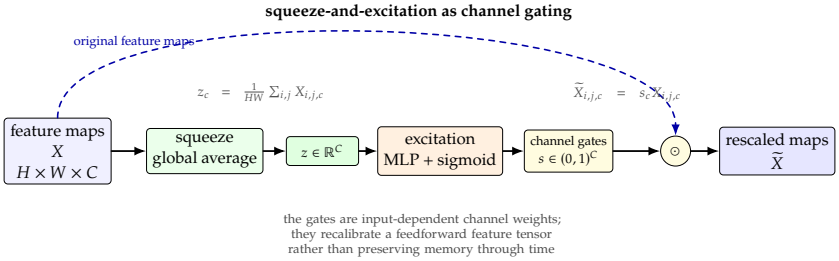

Figure 9.12: Squeeze-and-excitation block as a feedforward gate. The block summarizes each convolutional channel, computes sigmoid channel weights, and multiplies the original feature maps channel by channel.

Some activation functions can also be read through the same lens. ReLU can be written as a hard self-gated activation,

$$\text{ReLU}(a) = a \cdot \mathbf{1}[a > 0],$$

where the activation value is either passed through or suppressed. Swish replaces that hard indicator with a smooth sigmoid gate [225]:

$$\text{swish}_\beta(a) = a \cdot \text{sigmoid}(\beta a).$$

In this narrow sense, Swish is a self-gated activation: the scalar being transformed also produces its own gate. The analogy is helpful, but it should not be overstated. Swish has no separate learned control signal like $W_x x_t + W_h h_{t-1} + b$, no recurrent state, and no channel summary. It is an elementwise activation with a multiplicative interpretation.

Pause and compare

For the recurrent gate, the squeeze-and-excitation gate, and the Swish gate, identify what information produces the gate and what signal the gate modulates.

The practical lesson is that *gating* names a design pattern, not one specific layer. LSTMs and GRUs use gates to manage sequence state. Squeeze-and-excitation blocks use gates to recalibrate channels in a feedforward CNN. Swish uses the same multiplication idea inside a single activation value. Keeping those distinctions clear lets the analogy clarify architectures without pretending that all gates perform the same modeling role.

9.10 Dropout For Recurrent Networks

Dropout was defined in Chapter 5 as a stochastic regularizer with different training and evaluation behavior [263]. Here the sequence-specific question is where the mask is applied. Vertical dropout applies between layers, for example from embeddings to a recurrent layer or from one recurrent layer to the next. Horizontal, or recurrent, dropout applies along the recurrent computation itself. These choices are not interchangeable because the recurrent pathway is the pathway that carries state across positions.

Many successful recurrent language-model recipes apply dropout between stacked recurrent layers while leaving the recurrent state transition itself less disturbed [313]. Later work studied using the same dropout mask across time steps, often called variational or locked dropout in practice [63]. The reason is simple: if a new mask is sampled at every time step, the model sees a different random subnetwork at every token. That noise can make it harder for a state component to carry information consistently through time.

Figure 9.13 separates the main dropout locations and the choice between fresh and shared recurrent masks. This distinction matters because masking the path between layers is less disruptive than repeatedly perturbing the state pathway itself.

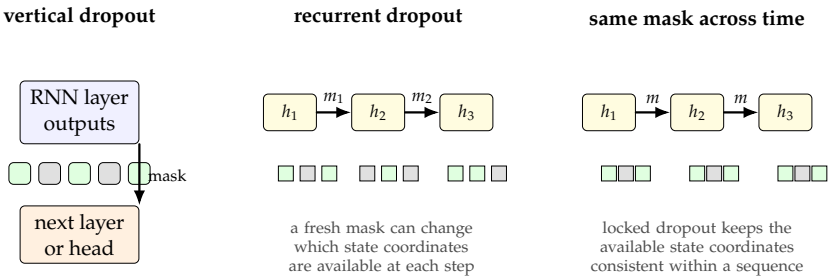

Figure 9.13: Dropout locations and mask-sharing choices for recurrent models. Vertical dropout masks connections between layers or before a classifier head; recurrent dropout masks the state pathway; same-mask recurrent dropout reuses one mask across time for a given sequence.

A numerical example shows the difference. Let a hidden vector be $h_t = (2, 4, 6)$ and use inverted dropout with keep probability $q = 0.5$. If the mask is $m = (1, 0, 1)$, the dropped vector is

$$\frac{m \odot h_t}{q} = (4, 0, 12).$$

With the same mask at positions $t = 1, 2, 3$, the second coordinate is consistently unavailable for that sequence during this training pass. With independently sampled masks, the available coordinates may change at every position. Both are forms of regularization, but the same-mask version better respects the idea that the recurrent state carries information through time.

Keras exposes both ordinary dropout and recurrent dropout arguments on recurrent layers, while other frameworks may require explicit modules or custom code. The beginner-safe policy is conservative. Start with no dropout or modest dropout after embeddings or between stacked recurrent layers. Add recurrent dropout only as a controlled experiment, because it can slow training and may harm memory. Always record the rate and location: input dropout, output dropout, between-layer dropout, or recurrent dropout.

The training/evaluation-mode distinction from Chapter 5 still matters. During recurrent training, inverted dropout divides kept activations by the keep probability so that their expected scale is similar to evaluation time. During evaluation, no units are dropped. A recurrent model with dropout must therefore be evaluated in the framework's evaluation mode; otherwise validation accuracy and final test accuracy are not measuring the intended model.

9.11 Normalization In Recurrent Networks

Batch normalization was defined in Chapter 5 for ordinary activation batches [113]. The recurrent question is whether those statistics are well defined for hidden states indexed by both example and time. A useful lab note is to ask which positions contributed to the statistic before trusting the layer name. In convolutional networks, batch normalization often works extremely well. In recurrent networks it is more delicate because activations differ by time step, sequences have different lengths, and padded positions can corrupt statistics if they are not excluded. A batch containing many short reviews padded to length 400 is not statistically equivalent to a batch of real 400-token reviews.

Specialized recurrent batch normalization methods exist [35]. Their existence is exactly why a beginner should not casually insert ordinary batch normalization into the recurrent state update and assume it behaves like the CNN case. One must decide which axes are normalized, whether statistics are time-specific, how masks are handled, and how training-time statistics relate to evaluation-time running statistics. Those choices are implementation details with mathematical consequences.

Layer normalization is often more natural for recurrent networks [5]. Instead of computing statistics across a mini-batch, layer normalization computes statistics across features within one example at one position. If a hidden vector is $h = (1, 3, 5)$, its mean is $\mu = 3$ and its variance is

$$\sigma^2 = \frac{(1-3)^2 + (3-3)^2 + (5-3)^2}{3} = \frac{8}{3}.$$

Ignoring the learned scale and shift and a small numerical ϵ, the normalized vector is approximately

$$\left(\frac{-2}{\sqrt{8/3}}, 0, \frac{2}{\sqrt{8/3}} \right) \approx (-1.225, 0, 1.225).$$

This computation does not require other examples in the batch, and it does not average over padded time positions.

Figure 9.14 shows the axis distinction. The layer-normalization calculation belongs to one valid hidden vector, while batch-normalization variants must define which batch and time positions provide statistics and how padding is excluded.

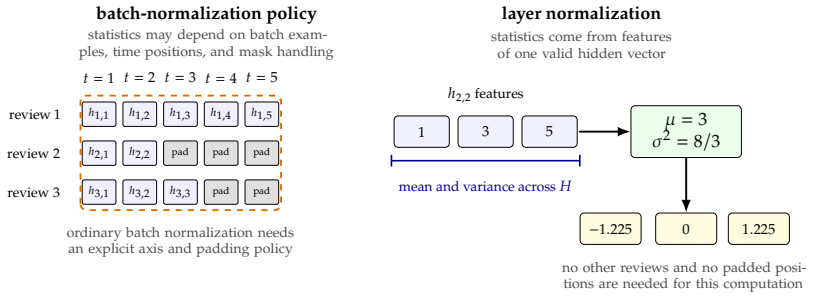

Figure 9.14: Normalization axes for recurrent hidden states. Batch normalization in a sequence model requires a policy for which batch and time positions supply statistics and how padded positions are excluded. Layer normalization computes statistics across the feature coordinates of one hidden vector, such as $h_{2,2} = (1, 3, 5)$, so it does not depend on other examples in the mini-batch.

The practical recommendation for this chapter is conservative. For a first IMDB recurrent classifier, do not add normalization inside the recurrent cell unless the chosen library layer provides a well-documented variant. If normalization is needed, prefer layer normalization or a layer-normalized recurrent cell. If batch normalization is used around sequence models, audit the axes, masks, padding positions, and training/evaluation behavior. As in Chapter 5, dropout before normalization also requires care because training-time random masking can change the distribution whose statistics are recorded.

9.12 fast.ai's AWD_RNN And AWD-LSTM Recipe

The previous sections treated recurrent architecture, regularization, and normalization as separate ideas: choose a simple RNN, GRU, or LSTM; decide where to place dropout; and be careful about normalization axes. The fast.ai text stack packages an important historical recipe into a small learner call. In modern fast.ai code, the public architecture symbol is `AWD_LSTM`; the underlying idea is often described more generally as an AWD-style recurrent neural network. The name comes from the AWD-LSTM language model of Merity, Keskar, and Socher: "ASGD Weight-Dropped LSTM" [180]. fast.ai uses this architecture as the recurrent body for both language modeling and text classification [52, 55].

The important lesson for this chapter is not the acronym. It is that a strong RNN result often comes from a coordinated recipe, not from replacing a single cell. The AWD-LSTM recipe combines embedding dropout, locked dropout on sequence activations, dropout between recurrent layers, weight dropout on the hidden-to-hidden LSTM matrices, gradient clipping in training recipes, and careful staged fine-tuning for transfer learning. Figure 9.15 places these pieces on the tensor path.

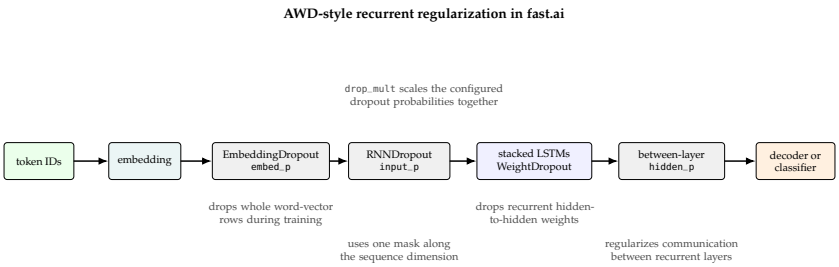

Figure 9.15: The fast.ai AWD-LSTM architecture combines several regularization locations instead of using one ordinary dropout layer. The public fast.ai learner examples pass `AWD_LSTM` as the architecture and often tune `drop_mult`, which scales the model's dropout settings together.

The fast.ai documentation exposes the most important dropout locations as configuration values such as `embed_p`, `input_p`, `hidden_p`, and `weight_p`. The learner helpers also accept `drop_mult`, a multiplier that scales the configured dropout rates together. This is convenient, but it changes several regularizers at once. A validation improvement after increasing `drop_mult` is evidence for that recipe setting, not evidence that one specific dropout location was responsible.

When I see a report that only says "I raised `drop_mult` and accuracy fell," I read it as a debugging lead rather than a conclusion. The multiplier may have removed useful signal from embeddings, recurrent activations, or hidden-to-hidden weights. A better follow-up keeps the split and learning rate fixed, records the actual dropout values, and inspects whether the model is underfitting before blaming AWD-LSTM itself.

The shortest classifier example mirrors the fast.ai tutorial style [56]. It is useful for seeing the API surface, but the comment marks the validation shortcut explicitly because it uses the official test folder as validation:

```
from fastai.text.all import *

path = untar_data(URLs.IMDB)
dls = TextDataLoaders.from_folder(path, valid="test")
# Tutorial shortcut only: do not use the official test folder for model
    selection.

learn = text_classifier_learner(
    dls,
    AWD_LSTM,
    drop_mult=0.5,
    metrics=accuracy,
)
learn.fine_tune(4, 1e-2)
```

Applying the validation/test-set rule from earlier experiment chapters, do not repeatedly use the official test folder as a validation set. The next version keeps validation inside the training data by using a data frame with an explicit validation flag. The code assumes a table with columns `text`, `label`, and `is_valid`; the same pattern works for a CSV exported from a cleaned training split.

```
from fastai.text.all import *
import pandas as pd

path = untar_data(URLs.IMDB_SAMPLE)
df = pd.read_csv(path / "texts.csv")

dls = TextDataLoaders.from_df(
```

```
    df,
    path=path,
    text_col="text",
    label_col="label",
    valid_col="is_valid",
    seq_len=72,
    bs=64,
)

learn = text_classifier_learner(
    dls,
    AWD_LSTM,
    drop_mult=0.5,
    metrics=accuracy,
)
learn.fine_tune(4, 1e-2)
```

Using the transfer-learning vocabulary from Chapter 7, ULMFiT first adapts a language model to the target corpus, saves its encoder, then loads that encoder into a classifier [106]. The recurrent text constraint is that the vocabulary must match, because the embedding rows only make sense for the token IDs they were trained with.

```
from fastai.text.all import *
import pandas as pd

path = untar_data(URLs.IMDB_SAMPLE)
df = pd.read_csv(path / "texts.csv")

dls_lm = TextDataLoaders.from_df(
    df,
    path=path,
    text_col="text",
    valid_col="is_valid",
    is_lm=True,
    seq_len=72,
    bs=64,
)
lm = language_model_learner(
    dls_lm,
    AWD_LSTM,
    metrics=[accuracy, Perplexity()],
    wd=0.1,
)
lm.fit_one_cycle(1, 1e-2)
lm.save_encoder("imdb_awd_encoder")

dls_clas = TextDataLoaders.from_df(
    df,
    path=path,
    text_col="text",
    label_col="label",
    valid_col="is_valid",
    text_vocab=dls_lm.vocab,
    seq_len=72,
    bs=64,
)
clas = text_classifier_learner(
    dls_clas,
    AWD_LSTM,
    drop_mult=0.5,
    metrics=accuracy,
)
```

```
clas = clas.load_encoder("imdb_awd_encoder")
clas.fit_one_cycle(1, 2e-2)
```

These examples are deliberately short, but they still need the same reporting discipline as the Keras recipe: validation split, sequence length, batch size, dropout multiplier, pretrained or not, epoch counts, learning rates, hardware, runtime, and whether the official test set was touched. fast.ai reduces the amount of code needed to train the model. It does not remove the need to define the experiment.

9.13 A Practical IMDB Sentiment Recipe

The companion script for this chapter is `code/chapter_recurrent_neural_networks/imdb_rnn_keras3.py`. It uses Keras 3 so that the model definition stays close to the beginner code in earlier chapters, and it can run on TensorFlow, PyTorch, or JAX backends when Keras supports the selected operations [134, 136]. The homework path uses the same shared raw-text IMDB protocol introduced in Chapter 8: build the vocabulary on the training split, reserve token ID 0 for padding, reserve token ID 1 for out-of-vocabulary tokens, and reuse the same maximum length and validation policy when comparing the average embedding baseline with recurrent models. The script still provides `-data-source keras-imdb` for legacy or quick comparisons with the Keras IMDB loader, which provides 25,000 training reviews and 25,000 test reviews as integer sequences with binary labels [135], but those runs should be reported separately because the preprocessing and split are different.

The first experiment should not be the most elaborate model. The run plan below fixes the comparison sequence before the code names one concrete implementation.

Run	Model	Test use
A	Avg. embeddings	no
B	Simple RNN	no
C	GRU	no
D	LSTM	no

Run A is the bag-of-embeddings-style baseline from Chapter 8, run B is a teaching baseline, and runs C and D are practical recurrent candidates. All four runs should use the same data source, vocabulary size, maximum sequence length, train-validation split, batch size, optimizer, and epoch budget. The comparison is then meaningful because the recurrent cell is the main changed ingredient. The official test set should be evaluated only after one model and training recipe have been selected by validation evidence, following the validation discipline used throughout the earlier experiment chapters; in the measured table below, that selected recipe is the length-400 GRU.

Measured run	Padding ratio	Val. acc.	Val. loss	Time	Interpretation
Average embeddings, len. 200	0.1944	0.8504	0.3628	8.02s	fast baseline
Simple RNN, len. 200	0.1944	0.8378	0.3992	181.78s	slower and weaker here
GRU, len. 200	0.1944	0.8590	0.3434	223.70s	stronger recurrent candidate
LSTM, len. 200	0.1944	0.8574	0.3822	220.83s	similar cost, no validation gain
GRU, len. 80	0.0201	0.8028	0.4315	109.08s	cheaper but truncates too much
GRU, len. 400	0.4711	0.8718	0.3126	420.36s	selected by validation despite cost
GRU recurrent dropout 0.2	0.1944	0.8328	0.3860	395.65s	slower and worse in this run

These measured artifacts make the applied point concrete. They were produced with the Keras IMDB loader before the shared raw-text homework protocol was standardized, so they are architecture and length-policy evidence rather than a cross-chapter leaderboard. The len. 400 GRU bought validation accuracy by spending about 1.9× the len. 200 runtime and padding nearly half the input slots; after that validation choice, a single locked test evaluation reported test accuracy 0.8658. The table is not a universal ranking of GRU and LSTM. It is evidence for this split, length policy, backend, hardware, and command record.

After that first table is stable, optional extensions should remain clearly labeled. A bidirectional recurrent encoder runs one GRU or LSTM left to right and another right to left, then usually concatenates their states, so a hidden width H in each direction produces $2H$ features for the next layer. This can be reasonable for classification because the whole review is available before prediction, but it is not a causal decoder for left-to-right generation. A fast.ai `AWD_LSTM` run is another follow-up, but it changes the regularization and training recipe. Treat either extension as a controlled experiment with its own runtime and validation record.

Figure 9.16 shows the tensor path used by the recurrent models in this run plan. The same path applies to a simple RNN, GRU, or LSTM; the recurrent cell changes, but the input shape, mask, selected final state, and classifier head stay comparable.

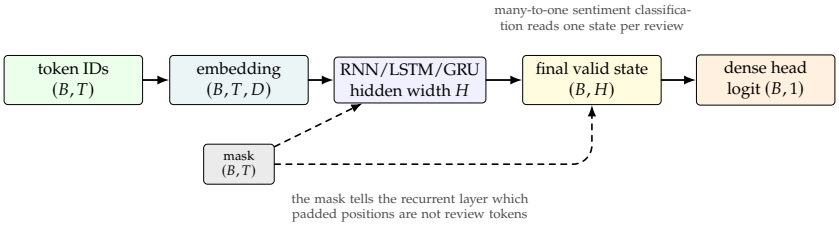

Figure 9.16: Tensor path for an IMDB recurrent sentiment classifier. Integer token IDs become embedding vectors, the recurrent layer reads the sequence with a padding mask, and the final valid hidden state feeds one binary sentiment logit.

The essential Keras model pattern is short:

```
inputs = keras.Input(shape=(max_length,), dtype="int32")
x = layers.Embedding(num_words, embedding_dim, mask_zero=True)(inputs)
x = layers.LSTM(hidden_size)(x)
outputs = layers.Dense(1)(x)
model = keras.Model(inputs, outputs)
```

The `mask_zero=True` argument carries the padding-mask principle from the embeddings chapter into the recurrent layer. The RNN-specific point is final-state correctness: a supported mask lets the layer ignore padded positions when it computes the state used for the sentiment logit. This does not remove the need to choose `max_length`; it only prevents padding inside that chosen length from behaving like ordinary words.

Keras mask rule. Token ID 0 is reserved for padding here, so real tokens should not be encoded as 0.

After activating the companion-code environment, the command below runs a small validation-only LSTM experiment. It records validation metrics but does not evaluate the held-out test set:

```
python imdb_rnn_keras3.py \
    --model lstm \
    --epochs 5 \
    --early-stopping-patience 2 \
    --batch-size 128 \
    --max-length 200 \
    --num-words 10000 \
    --data-source shared-imdb \
    --data-dir ../chapter_embeddings/data \
    --save-artifacts
```

The early-stopping option uses a validation-loss callback and restores the best weights when it stops training [132]. After model selection, add `-evaluate-test` once for the chosen run. The script also provides `-quick` and `-synthetic-data` modes for environment checks that do not require IMDB files. Those modes are useful for verifying dependencies and tensor shapes, but their accuracy is not evidence about movie-review sentiment.

The result record for each run should include the command, seed, backend, hardware, package versions, maximum length, vocabulary size, padding policy, parameter count, epoch count, best validation loss, best validation accuracy, elapsed time, and whether the test set was evaluated. With `-save-artifacts`, the companion script writes the training history to CSV and the run summary to JSON. This is not bookkeeping for its own sake. Without these fields, a later accuracy comparison may confuse model quality with a larger vocabulary, longer input length, or different accelerator.

For fast iteration, use a bounded run first: smaller training subset, fewer epochs, and synthetic data if the purpose is only to check the environment. For accuracy evidence, use real IMDB data and a fixed validation split. If validation accuracy improves but training time doubles, report both. Applied deep learning is not just the largest number in the accuracy column; it is the tradeoff between correctness, speed, reproducibility, and the cost of the next experiment.

9.14 Accuracy And Speed Checklist

A recurrent sentiment model has several levers, and changing too many at once makes the result hard to interpret. Start with a maximum length chosen from the length distribution, not from habit. Use a vocabulary size that fits the assignment budget, such as 10,000 or 20,000 tokens for a first IMDB run. Keep the embedding dimension and hidden width moderate until the pipeline is correct. A model that reaches a reasonable validation score quickly is more useful for learning than a large model that makes each comparison expensive.

Padding should be measured. If most token slots in a batch are padding, try a shorter maximum length or bucketed batching. If the validation score collapses after shortening the maximum length, important evidence may have been truncated. This is an empirical tradeoff, and the chapter's running example makes it visible: the GRU len. 80 run had little padding, 0.0201, and finished in 109.08s, but validation accuracy fell to 0.8028. The GRU len. 400 run padded much more, 0.4711, and took 420.36s, but reached 0.8718 validation accuracy. Long reviews may contain sentiment reversals that do not appear near the beginning, so padding waste and truncation error have to be compared with actual validation evidence.

Use masks or packed sequences whenever padding is present. Verify that the model uses the final valid state rather than the state after padded positions. In Keras, this usually means checking that the embedding or masking layer propagates a mask to a recurrent layer that supports it. In direct PyTorch code, it often means carrying lengths into packed sequences or manually gathering the state at length minus one.

Use gradient clipping for recurrent models unless there is a reason not to. It is a low-cost guard against exploding gradients. Use early stopping or at least save the best validation epoch, because recurrent models can overfit. Tune dropout as a controlled experiment, and record whether it is input, output, between-layer, or recurrent dropout. Prefer layer normalization over casual batch normalization inside recurrent computations.

Finally, compare model families under the same budget. A useful result table reports accuracy and speed together: average embeddings may be fastest, simple RNN may be educational but weak, GRU may be a strong speed-accuracy compromise, and LSTM may be more flexible. The conclusion should come from validation curves and a locked final test evaluation, not from the reputation of the architecture.

Run	Max len.	Params	Padding	Val. acc.	Time
Average embeddings	200	640,065	0.1944	0.8504	8.0s
Simple RNN	200	648,321	0.1944	0.8378	181.8s
GRU	200	665,025	0.1944	0.8590	223.7s
LSTM	200	673,089	0.1944	0.8574	220.8s
GRU, longer input	400	665,025	0.4711	0.8718	420.4s

These rows come from the committed Keras IMDB run summary for the chapter. For the homework comparison with Chapter 8, rerun the same model families on the shared raw-text IMDB split and report that source explicitly. The selected GRU length-400 run was then evaluated once on the locked test set, reaching 0.8658 test accuracy. The table makes the tradeoff visible: the longer input improved validation accuracy, raised runtime from 223.7s to 420.4s, and more than doubled padding waste compared with the length-200 GRU.

9.15 Summary

An RNN repeatedly applies a shared state-update function along a sequence. The loop diagram becomes an unrolled computation graph during training, and backpropagation through time computes gradients through that graph. Stacked RNNs track both layer depth and sequence time.

The simple Elman RNN defines the core recurrence, but long dependencies stress gradients. Clipping helps exploding gradients; LSTM and GRU cells address vanishing gradients with gates and additive state paths. Peephole LSTMs, GRUs, feedforward gates in CNNs, and self-gated activations show that gating is a general neural-network tool, not only an RNN trick.

Sequence data adds pipeline risks: padding, masks or lengths, collation, bucketing, dropout, normalization, and bidirectionality all change what the model can use. fast.ai's AWD-style LSTM recipe packages several regularization choices into a strong applied recurrent model. The practical recipe is to start simple, log tensor shapes and lengths, compare baselines, keep validation and test roles separate, and report accuracy with runtime. Before changing the cell, inspect the sequence lengths and padding ratio; otherwise the experiment may be optimizing mostly empty token slots.

Chapter 10 keeps the sequence problem but shortens the information path: instead of forcing every output to depend on one recurrent state chain, attention lets a query read directly from relevant sequence positions.

9.16 Exercises

These exercises use sequence arithmetic as a diagnostic tool. Padding waste, masks, and truncation are not just implementation details; they decide whether the model is reading the intended tokens or paying for empty slots.

Exercise 1. A batch of embedded reviews has shape $(64, 150, 32)$. Identify the batch size, maximum sequence length, and embedding dimension. If a recurrent layer has hidden width 80 and returns all hidden states, what

is the output shape? If a second recurrent layer with hidden width 40 is stacked above it, what shape does that second layer receive, and what shape does it return when it returns all hidden states?

Exercise 2. Let $h_t = \tanh(0.8x_t + 0.4h_{t-1})$, $h_0 = 0$, and $(x_1, x_2, x_3) = (1, -1, 0.5)$. Compute h_1, h_2, and h_3 to three decimal places.

Exercise 3. For a simple RNN with input dimension D and hidden width H, derive the parameter count $H(D+H)+H$. Then compute the count for $D = 50$ and $H = 100$.

Exercise 4. Unroll a simple RNN for four time steps. In words, explain why the four copies of the cell do not have four independent recurrent weight matrices.

Exercise 5. Pad the three sequences $(8, 4, 9)$, (3), and $(7, 7)$ with pad token 0. Write the padded batch and the corresponding mask.

Exercise 6. Two possible batches have sequence lengths $(100, 10, 8, 7)$ and $(100, 98, 96, 94)$. Compute the padding waste for each batch after padding to the longest sequence in that batch.

Exercise 7. Explain vanishing and exploding gradients using the quantities 0.7^{20} and 1.2^{20}. What practical method helps with exploding gradients?

Exercise 8. In a one-dimensional LSTM, suppose $c_{t-1} = 2$, $f_t = 0.75$, $i_t = 0.1$, $g_t = -1$, and $o_t = 0.5$. Compute c_t and h_t.

Exercise 9. Ignoring separate framework-specific bias conventions, compare the parameter counts of a simple RNN, GRU, and LSTM when $D = 16$ and $H = 32$. Which cell has the fewest recurrent parameters, and why?

Exercise 10. In fast.ai's `AWD_LSTM` recipe, name three different regularization locations. Explain why changing `drop_mult` is not the same as changing one ordinary dropout layer in isolation.

9.17 Homework: IMDB Sentiment Classification With An RNN

Train and evaluate a recurrent neural network for binary sentiment classification on the same shared IMDB movie-review dataset used in the embeddings homework. Reuse the same raw-text source, split seed, tokenization rule, vocabulary size, maximum sequence length, and validation policy unless your controlled experiment is explicitly about one of those quantities. The Keras IMDB loader is acceptable for a quick compatibility check, but it should not be mixed into the final bag-of-embeddings-versus-RNN comparison because it changes preprocessing and the split. The goal is not only to obtain a high validation accuracy, but also to demonstrate that the data pipeline, masking, model comparison, and test-set protocol are correct. Your submission must include a non-recurrent baseline and at least one recurrent model. The baseline may average token embeddings or use another clearly described order-insensitive representation. The recurrent model must be a GRU or LSTM. You may also include a simple RNN as a teaching comparison. All controlled comparisons must use the same train-validation split, maximum sequence length, vocabulary size, batch size, and epoch budget unless the comparison is explicitly about one of those quantities.

Report the following fields for every run: command or notebook cell, data source, random seed, framework and backend, hardware, vocabulary size, maximum sequence length, padding policy, whether masks or packed sequences were used, model type, embedding dimension, hidden width, parameter count, optimizer, learning rate, gradient clipping setting, dropout setting, number of epochs, best

validation loss, best validation accuracy, runtime, and whether the test set was evaluated. The official test set should be evaluated once, after selecting the final model by validation evidence.

Include at least one controlled experiment beyond the required baseline-versus-recurrent comparison. Examples include GRU versus LSTM, simple RNN versus GRU, dropout off versus dropout on, a fast.ai `AWD_LSTM` dropout-multiplier comparison, maximum length 100 versus 200, or unbucketed versus bucketed batching. Interpret the result in words. If the faster model is slightly less accurate, discuss the tradeoff instead of hiding it.

The written report should contain a short description of the data representation, a table of runs, training and validation curves if available, the final test result for the selected model, and one debugging note. The debugging note should name a concrete issue you checked, such as an incorrect padding mask, a wrong validation split, a threshold applied to logits as though they were probabilities, or accidental repeated use of the test set.